

MDP Cloud Migration Technical Requirements

Introduction to MDP

MDP manages membership, eligibility, and demographic data, assigning each person a unique MMI ID for fraud detection and data sharing. Algorithms and Blue plan inputs create the MMI.

The MMI ID streamlines claims processing by accurately identifying members and reducing duplicates. It aids care management by helping Plans track patient histories and coordinate services. The ID enables advanced analytics for population health insights and improves decision-making. It also ensures consistent member identification is needed for regulatory compliance.

1. Overview

- Purpose: Modernize the MDP application by migrating it to AWS Cloud.
- Scope: MMI assignment and services still on-premises, including UI and reporting.
- Stakeholders include Business, Architecture, Project Management, and Application Owners.

2. Functional Requirements

- System should continue to make MMI assignments and be able to respond to real time requests.
- Review processes to identify areas to optimize and streamline.
- Migrate the ETL code (MMI & MMI-Services Roster Extracts) from DataStage/Unix to Databricks native components using a conversion tool if possible, i.e. Lakebridge.
- Develop Quality Stage equivalent rules with PySpark code and AWS native tools for fuzzy logic processing. (matching or better than existing data quality)
- Migrate DB2 database objects and data to Databricks.
- Perform historical data migration using MFT exports from DB2, Databricks load operations, or existing DMS jobs implemented with HUDI (internal software).
- Retain the current structure of MMI Identifiers and current MMI IDs should not change
- Assess and implement a replacements for the current reporting tool. Our preference is for native reporting tools where possible.
- Design and develop a CI/CD pipeline for the modernized application.
- Optimize the performance and cost of the new code to align with business SLAs.

- Redesign, update, and integrate the MMI UI into the BSI portal.
- Enhance system throughput and user experience as stated in BRD
- Conduct PV, PVE, and PE testing with sufficient synthetic data volumes to meet application requirements.
- Execute parallel runs and data validation in production over two cycles between legacy and modernized cloud-based MMI applications.
- Maintain code and data synchronization during ongoing development and maintenance in the first three quarters of 2026 for the legacy application.
- Set up AWS environments for Dev, PV, PVE, PE, and PROD.
- Comparison and validation of data between on-prem and cloud
- Data standardization and cleansing for input to processes and track where changes are made.

3. Non-Functional Requirements

- Performance: Support capacity for up to 1 billion MMI IDs (currently approximately 500 million).
- UI requirements: Accommodate concurrent users with response times under one second.
- Security: Tokenize data with Protegrity and encrypt with PGP; ensure all access complies with Enterprise EDS standards.
- Scalability: Leverage AWS auto scaling groups and load balancing to dynamically increase or decrease computing resources based on demand, ensuring efficient horizontal expansion across cloud infrastructure.
- Availability: Ensure a minimum monthly uptime of 99.9% by leveraging AWS and Databricks cloud-native features. Optimize resource usage and scalability to efficiently handle large-scale data volumes, fully utilizing the managed services and auto-scaling capabilities available within these platforms.

4. Technical Requirements and Architecture

Current Technologies:

- IBM DataStage and QualityStage
- Linux servers
- Java for user interface
- Tableau and SQL Server Database for reporting
- IBM DB2
- Python

- Unix scripts
- Protegrity Tokenizing software
- PGP encryption
- MFT
- Jira and ServiceNow
- GitHub

Cloud Migration Technologies:

- AWS (EC2, DMS, Lambda)
- Java for UI
- Databricks
- Airflow
- PySpark / SQL
- Python
- Cloud based tool for matching/ Fuzzy logic
- Tableau or similar cloud reporting tool
- Protegrity Tokenization
- PGP encryption
- MFT
- Jira, ServiceNow
- CI/CD pipelines
- REST API to Acxiom Vendor
- Git Hub

6. Acceptance Criteria and Testing

- Use an Agile approach with EPICs as functional modules.
- Release cycles should align with EPICs.
- Ensure comprehensive integration of all system modules.
- Conduct unit, integration, regression, UAT, and performance testing.
- Provide detailed technical design documents for each module.

7. Assumptions, Constraints, and Risks

- Project begins Q1 2026, and targets go-live by November 2026.

- Payments are tied to SME approval of individual epics.
- Delivery delays will incur maintenance and licensing costs for the on-prem application.
- Identify risks and propose mitigation strategies promptly.

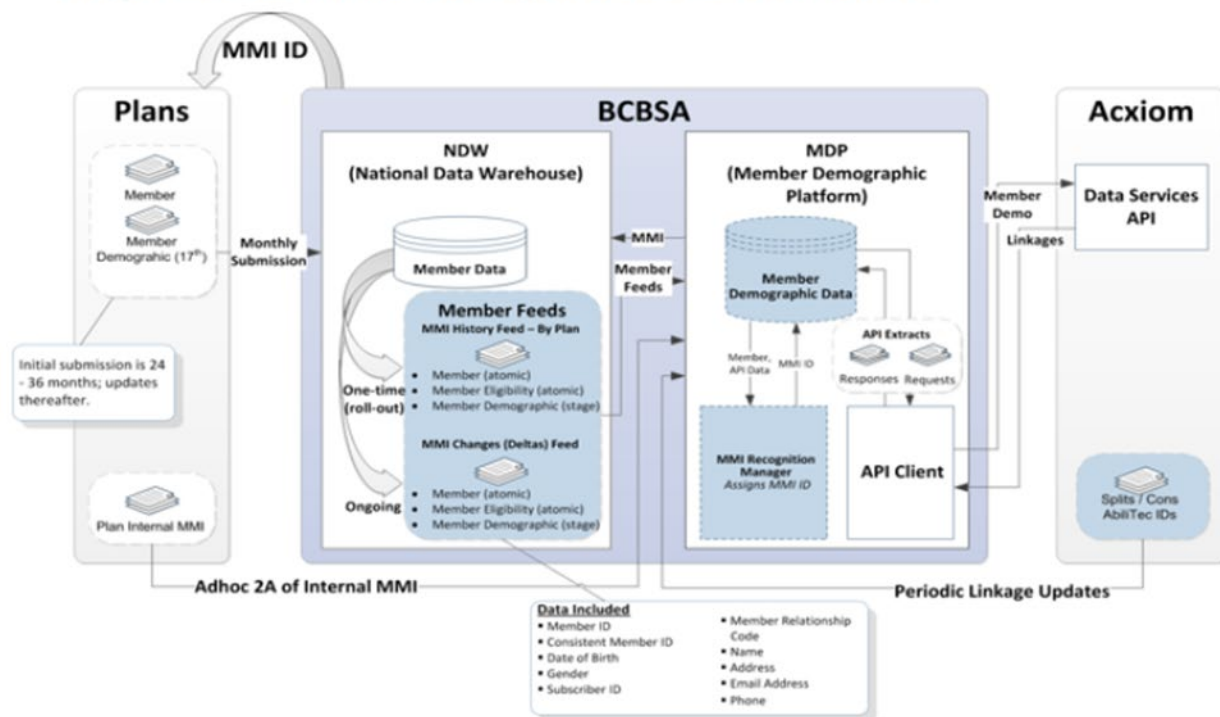
8. Inventory

Current on-prem MMI project overview:

- 550 Datastage Parallel jobs/sequences
- 2 Quality stage jobs, 14 Matching rules
- 100 Unix scripts
- 160 DB2 tables, 30 views, 31 sequences
- 5 procedures, 1 trigger
- 27 Tableau reports
- 5 UI screens
- 10 TB historical data

9. Current Architecture Data Flow

High Level MMI Assignment Data Flow



10. EPICS and Key Milestones

- History Data copy from on-prem to Cloud
- Staging & Integration
- Acxiom API calls
- Full Recognition Module – includes Pre-RM, DQ, Post-RM, Matching rules using Fuzzy logic and MMI assignment
- Accuracy
- Reconciliation (Over/Under, all other extracts, MDPR UI Refresh), UI development
- Reporting Tier
- Data visualization reports
- UI development
- Outbound File extracts (MMI-Services Extracts, over under, Detail, New member, BHI, VBP & Rosters)
- REL & BIFSG
- On -Prim enhancement catchup
- Integration testing
- Parallel run and data validation between cloud and non-cloud
- Run guide and Additional operational documents
- Member Data Sharing